Java SE 17 Programmer II（1Z0-826-JPN）　試験内容 6 ファイル I/O　#15

PathとFiles

Path の分解・resolve・relativize・normalize、Files の作成・コピー・削除・読み書き、walk・list・find・属性

各ページの下に、根拠にした資料（Oracle の試験内容チェックリスト、Java SE 17 API の java.nio.file パッケージ（Path・Paths・Files・StandardCopyOption・StandardOpenOption）と BasicFileAttributes、The Java Tutorials の File I/O）を書いた。コードの結果は JDK 17（Eclipse Temurin 17.0.21、Linux）で実行して確かめた

全体像 Path は「場所」、Files は「操作」

Path
Path.of・Paths.get で作る。不変。ファイルがあるかは調べない

↓ 引数に渡す

Files
static メソッド。ファイルシステムに触る。IOException を投げる

↓ 失敗の理由ごとの例外

NoSuchFileException・FileAlreadyExistsException・DirectoryNotEmptyException
どれも IOException のサブクラス

java.io	java.nio.file
File	Path ＋ Files（file.toPath()・path.toFile()）
FileNotFoundException	NoSuchFileException
new BufferedReader(new FileReader(f))	Files.newBufferedReader(p)
file.listFiles()	Files.list(p)・walk(p)

出典：API java.nio.file パッケージ・Path・Files／The Java Tutorials「Legacy File I/O Code」

第1章

Path の操作

分ける、つなぐ、すっきりさせる。

パスを分ける

/	home	user	docs	a.txt
ルート	0	1	2	3

```
Path p = Path.of("/home/user/docs/a.txt");
p.getFileName()   // a.txt
p.getParent()     // /home/user/docs
p.getRoot()       // /
p.getNameCount()  // 4（ルートは数えない）
p.getName(0)      // home
p.subpath(1, 3)   // user/docs（3 は含まない）
p.getName(4)      // IllegalArgumentException
```

```
Path r = Paths.get("docs", "x", "b.txt");
r                 // docs/x/b.txt
r.getRoot()       // null（相対パス）
r.isAbsolute()    // false

Path.of("/").getFileName()  // null
Path.of("/").getParent()    // null
Path.of("/").getNameCount() // 0
```

> Path.of("a","b").equals(Path.of("a/b")) は true、
> Path.of("a/./b").equals(Path.of("a/b")) は false。

出典：API Path・Paths／JDK 17 での実行結果

つなぐ・比べる

```
Path b = Path.of("/a/b");
b.resolve("c/d")        // /a/b/c/d
b.resolve("/x/y")       // /x/y（絶対パスはそのまま）
b.resolveSibling("z")   // /a/z

Path.of("/a/b").relativize(Path.of("/a/c/d"))  // ../c/d
Path.of("a/b").relativize(Path.of("a"))        // ..
Path.of("/a").relativize(Path.of("b"))
// IllegalArgumentException: 'other' is different type of Path

b.resolve("c"); b   // /a/b（不変、元は変わらない）
```

```
Path.of("/a/./b/../c").normalize()  // /a/c
Path.of("../x/./y/..").normalize()  // ../x
Path.of("a/../..").normalize()      // ..

Path p = Path.of("/a/b/c.txt");
p.startsWith("/a")      // true
p.startsWith("/a/b/c")  // false
p.endsWith("c.txt")     // true
p.endsWith("txt")       // false（要素の単位）
```

> startsWith・endsWith は文字列の比較ではなく、名前の要素で比べる。

出典：API Path.resolve・resolveSibling・relativize・normalize・startsWith・endsWith／JDK 17 での実行結果

第2章

Files で作る・写す・消す

どんなときに、どの例外か。

作る・消す

メソッド	親がない	もうある
createDirectory	NoSuchFileException	FileAlreadyExistsException
createDirectories	親も作る	何もしない

メソッド	ない	中身のあるディレクトリ
delete	NoSuchFileException	DirectoryNotEmptyException
deleteIfExists	false を返す	DirectoryNotEmptyException

```
Path x = d.resolve("x/y");
Files.createDirectory(x);
// NoSuchFileException（x がない）
Files.createDirectories(x);   // x/y を作る
Files.createDirectories(x);   // 2 回目も OK
Files.createDirectory(x);
// FileAlreadyExistsException
Files.delete(d.resolve("x"));
// DirectoryNotEmptyException
```

出典：API Files.createDirectory・createDirectories・delete・deleteIfExists／JDK 17 での実行結果

写す・移す・オプション

```
Files.copy(f, g);   // g がない → OK
Files.copy(f, g);   // FileAlreadyExistsException
Files.copy(f, g, StandardCopyOption.REPLACE_EXISTING);

Files.move(g, d.resolve("h.txt"));
// exists(g) false、exists(h.txt) true

Files.copy(dir, dir2);
// dir2 は空のディレクトリ（中身は写さない）
```

StandardCopyOption：REPLACE_EXISTING・COPY_ATTRIBUTES・ATOMIC_MOVE。

StandardOpenOption	中身
（なし）	なければ作り、あれば中身を消して書く
CREATE_NEW	新しく作る。あれば FileAlreadyExistsException
APPEND	後ろに足す

```
// g の中身 "l1\nl2\nl3"（最後に改行なし）
Files.write(g, List.of("A","B"), APPEND);
Files.readAllLines(g)  // [l1, l2, l3A, B]
```

出典：API Files.copy・move・write、StandardCopyOption・StandardOpenOption／JDK 17 での実行結果

調べる・比べる

メソッド	戻り値
exists・notExists	boolean（調べられないと両方 false）
isDirectory・isRegularFile	boolean
isSameFile(a, b)	同じファイルを指すか
mismatch(a, b)	最初にちがう位置（同じなら -1）
size	long（バイト数）

```
// f="l1\nl2\nl3"（8 バイト）
// h="l1\nl2\nl3A\nB\n"
Files.isSameFile(f, d.resolve("./f.txt")) // true
Files.mismatch(f, h)   // 8（短いほうの大きさ）
Files.size(f)          // 8
```

> Path.equals はファイルを見ない。同じファイルか調べるのは Files.isSameFile。

出典：API Files.exists・notExists・isSameFile・mismatch・size／JDK 17 での実行結果

第3章

読み書きと走査・属性

List か Stream か、どこまでたどるか。

読む・書く

メソッド	戻り値
readAllLines(p)	List<String>
lines(p)	Stream<String>（閉じる）
readString(p)	String（Java 11〜）
write(p, 行のリスト)	Path（各行の後ろに改行）
writeString(p, s)	Path
newBufferedReader(p)	BufferedReader
newBufferedWriter(p)	BufferedWriter

```
List<String> l = Files.lines(Path.of("x"));
// × incompatible types: Stream<String>
//   cannot be converted to List<String>

// L.txt="aa\nbbb\nc\n"
try (Stream<String> s = Files.lines(L)) {
  s.filter(x -> x.length() > 1).toList();
}   // [aa, bbb]

Files.readAllLines(none)
// NoSuchFileException
```

> 文字コードの既定は UTF-8。

出典：API Files.readAllLines・lines・readString・write・writeString・newBufferedReader／JDK 17 でのコンパイルと実行の結果

たどる

```
d
├ 3.md        // 3 バイト
└ a
   ├ 1.txt    // 1 バイト
   └ b
      └ 2.txt // 2 バイト
```

呼び方	結果（d からの相対）
walk(d)	[（空＝d 自身）, 3.md, a, a/1.txt, a/b, a/b/2.txt]
walk(d, 1)	d・3.md・a の 3 つ
list(d)	[3.md, a]（自分は含まない）

```
try (Stream<Path> s = Files.find(d, 10,
    (p, attr) -> attr.isRegularFile()
                 && attr.size() >= 2)) { … }
// [2.txt, 3.md]

BasicFileAttributes at = Files.readAttributes(
    d.resolve("3.md"), BasicFileAttributes.class);
at.size() at.isDirectory() at.isRegularFile()
// 3 false true
at.lastModifiedTime()  at.creationTime()
```

> find の条件は BiPredicate<Path, BasicFileAttributes>。walk・list・find の Stream は閉じる。

出典：API Files.walk・list・find・readAttributes、BasicFileAttributes／JDK 17 での実行結果

本番での問われ方

ねらい	見るところ
Path の分解	ルートは数えない、番号は 0 から、subpath の終わりは含まない
resolve・relativize	絶対パスの resolve は引数そのもの、種類ちがいの relativize は例外
normalize・startsWith	先頭の .. は残る、要素の単位で比べる、Path は不変
例外	NoSuchFile・FileAlreadyExists・DirectoryNotEmpty の使い分け
読み書き	readAllLines は List、lines は Stream、APPEND・CREATE_NEW
走査	walk は自分を含む、list はすぐ下、find は BiPredicate

出典：試験内容チェックリスト 6／本資料の問題演習（全 30 問、JDK 17 で確認）